DMPA: Model Poisoning Attacks on Decentralized Federated Learning for Model Differences

Anonymous submission

Abstract

Federated learning (FL) has garnered significant attention as a prominent privacy-preserving Machine Learning (ML) paradigm. Decentralized FL (DFL) eschews traditional FL's centralized server architecture, enhancing the system's robustness and scalability. However, these advantages of DFL also create new vulnerabilities for malicious participants to execute adversarial attacks, especially model poisoning attacks. In model poisoning attacks, malicious participants aim to diminish the performance of benign models by creating and disseminating the compromised model. Existing research on model poisoning attacks has predominantly concentrated on undermining global models within the Centralized FL (CFL) paradigm, while there needs to be more research in DFL. To fill the research gap, this paper proposes an innovative model poisoning attack called DMPA. This attack calculates the differential characteristics of multiple malicious client models and obtains the most effective poisoning strategy, thereby orchestrating a collusive attack by multiple participants. The effectiveness of this attack is validated across multiple datasets, with results indicating that the DMPA approach consistently surpasses existing state-of-the-art FL model poisoning attack strategies.

Introduction

The advent of diverse privacy regulations has significantly increased the need for privacy preservation in Machine Learning (ML). Federated Learning (FL) emerges as a novel collaborative and privacy-preserving ML paradigm that has attracted substantial attention from both academic and industrial communities (McMahan et al. 2017). FL enables participants to share model parameters instead of raw data, thereby ensuring data privacy (Yang et al. 2019). Traditional FL architectures rely on a central server to distribute, called Centralized FL (CFL), receive and aggregate model parameters from participants (Alazab et al. 2021). However, this client-server architecture suffers from significant drawbacks, including susceptibility to a single point of failure and server-side bottleneck (Martínez Beltrán et al. 2023). To address these challenges, Decentralized FL (DFL) has been introduced.

In DFL, each client independently manages the processes of establishing communication, sharing, and aggregating model parameters with other clients (Lian et al. 2017). During each iteration, participants execute several tasks: training their local models, transmitting model parameters to directly connected peers, aggregating received parameters, and updating their local models accordingly. Unlike CFL, clients in DFL have the autonomy to independently select any other participant to establish bidirectional communication connections, allowing for customizable overlay network topology, such as fully connected, ring, star, and even dynamic configurations (Yuan et al. 2024). This adaptability enables DFL to effectively address the single point of failure issue inherent in CFL, thereby enhancing the system's robustness and scalability. Nevertheless, this decentralized nature introduces new security vulnerabilities, making DFL more susceptible to various threats, including model poisoning attacks.

Model poisoning attacks involve malicious clients directly altering their local model parameters to negatively influence the global model training by sending harmful updates (Feng et al. 2023). In FL, since participants can directly share and modify model parameters, model poisoning attacks are relatively more straightforward to execute and pose significant threats. Current model poisoning attacks primarily target CFL models. These adversarial clients transmit meticulously crafted local model updates to the central server during the training phase, leading to issues such as reduced accuracy of the global model (Cao and Gong 2022). These attacks presuppose that the attacker controls a substantial number of compromised real clients, which may include either hijacked clients or fabricated ones.

Nevertheless, the architectural distinctions between DFL and CFL, particularly in overlay network topology, imply that existing attacks targeting CFL can not be directly transferable to DFL. In DFL, the communication links are significantly longer compared to CFL, making it challenging for a malicious participant to impact the entire federation (Martínez Beltrán et al. 2023). Additionally, in DFL, each node could decide whether to disseminate and receive models from neighboring nodes. This capability enables the potential blockage of malicious attacks at intermediate nodes, thereby diminishing the effectiveness of such attacks. Consequently, from an adversarial perspective, it is imperative to develop model poisoning attacks that are effective in DFL.

To this end, this paper investigates model poisoning attacks within DFL and proposes the Decentralized Model Poisoning Attack (DMPA) based on an Angle Bias Vector

to rectify reversed model parameters. Specifically, by determining the eigenvalues of the compromised benign parameters, the method extracts the corresponding eigenvectors to compute the angular deviation vector. This vector is derived by exploiting the discrepancies between models and ultimately adjusting the negative parameters, thereby generating an effective attack model across the majority of participants. The main contributions of this paper are: *(i)* design and implementation of a model poisoning attack, called DMPA, to compromise the model robustness of DFL models; *(ii)* an extensive series of experiments are conducted to evaluate the proposed DMPA and compared with selected state-of-the-art attack algorithms under diverse robustness aggregation functions, which encompass three benchmark datasets (MNIST, Fashion-MNIST, and CIFAR-10) and three varying overlay topologies (fully connected, star, and ring); and *(iii)* critical insights into the robustness of the DFL model concerning both offensive and defensive points of view. The experiments demonstrate that the DMPA presented in this work exhibits a more potent attack capability and broader propagation potential, effectively compromising the DFL system.

The remainder of this paper is structured as follows. Section 2 provides an overview of the underlying security issues in DFL and discusses the concept of model poisoning attacks. Subsequently, Section 3 formalizes the problem. Section 4 delineates the design specifics of DMPA, the proposed model poisoning attack. Section 5 then evaluates DMPA's performance in comparison to selected related works. Finally, Section 6 offers a summary of the contributions made by this research and outlines potential avenues for future investigation.

Background and Related Work

This section provides an introduction to the security problem in DFL and a summary of the current research on model poisoning attacks in FL.

Security Problem in DFL

Differing from traditional CFL, neighboring clients in DFL exchange local model parameters or gradients over a P2P network and construct consensus models independently. DFL solves the problem of risk associated with a single point of failure, enhances its scalability, and is well suited for applications in the Industrial Internet of Things (IIoT) (Tan et al. 2023). However, the decentralized nature of DFLs rather increases their risk of being exposed to malicious attacks, especially poisoning attacks (Feng et al. 2024). Poisoning attacks, which aim to diminish the resilience of FL models, can be classified into data poisoning and model poisoning attacks. Data poisoning attacks typically involve malicious clients interfering with the training process by introducing harmful data (such as backdoor attacks or label flipping) into the local training dataset, thereby inducing biased learning outcomes (Hallaji, Razavi-Far, and Saif 2022). Conversely, model poisoning attacks are characterized by malicious clients directly altering their local model parameters and affecting the global model training via harmful model updates. To improve the model robustness of FL and defend against the poisoning attacks, (McMahan et al. 2017) proposed the use of averaging model parameters to improve training efficiency. (Blanchard et al. 2017) introduced the Krum method, which excludes malicious updates by calculating the Euclidean distance of a multi-node model and selecting the model with the smallest distance. This method reduces the impact of malicious attacks and is now widely used for robust aggregation in DFL. (Yin et al. 2018) developed two robust distributed learning algorithms for Byzantine errors and potential adversarial behavior, a robust distributed gradient descent algorithm based on Median and Trimmed Mean operations, respectively. These two methods are widely used in both CFL and DFL because they can be implemented with only one communication round and achieve optimal model robustness.

Model Poisoning Attack

Compared with data poisoning attacks, model poisoning attacks are easier to execute as they directly modify the shared model. Therefore, there has been research suggesting how to optimize the attack method to enhance the attack effectiveness. Existing model poisoning attacks mainly target CFL. The attack assumes that the attacker has access to a large number of compromised clients. During the training process, malicious clients send carefully designed local model updates to the server, resulting in problems such as a decline in the accuracy of the global model (Cao and Gong 2022). The MIN-MAX and MIN-SUM methods proposed by (Shejwalkar and Houmansadr 2021) respectively minimize the maximum and sum of the distance between toxic updates and all benign updates in CFL. These methods assume that malicious model updates are the sum of aggregated model updates and a fixed disturbance vector scaled by factors before the attack, and implement the attack by maximizing the distance between toxic updates and benign updates in the inverse direction of the global model. The "a little is enough" (LIE) attack (Baruch, Baruch, and Goldberg 2019) directly modifies the model parameters, which generates malicious model updates in each training round by calculating the average of the real model updates of the malicious client and perturbing them.

Most model poisoning attacks rely on additional information from the central server, such as aggregation methods, global models, or even the training data utilized by nodes. This dependency renders external attacks impractical. Nevertheless, some attacks are engineered to maintain efficacy even in the absence of such additional knowledge. For instance, (Zhang et al. 2023) employs global historical data to build an estimator that forecasts the subsequent round of global models as a benign reference. Despite this, the approach necessitates substantial resources and encounters difficulties in accessing the global model within DFL systems, thereby limiting its practical application in DFL.

To conclude, while a substantial amount of research has been dedicated to optimizing model poisoning attacks against the FL, these efforts predominantly concentrate on the CFL paradigm, with minimal attention given to the DFL paradigm. Furthermore, the current attack methodologies

exhibit limitations, such as inconsistent efficacy and susceptibility to detection by defensive mechanisms. To address these research gaps, this paper presents the design and implementation of a novel attack strategy tailored for DFL. This proposed attack demonstrates broad effectiveness across various datasets, diverse ML model architectures, and multiple types of DFL overlay topologies.

Problem Setup

DFL Training Process

In the DFL framework, consider a network of K clients, each denoted as $k \in \{1, 2, \ldots, K\}$. In each communication round, each client k maintains and updates a local model w_t^k according to an aggregation algorithm $A(\cdot)$ defined by this framework. The communication between clients can be represented as a graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes (clients) and $\mathcal{E}$ is the set of edges (communication links) between the nodes. The neighborhood of a client k, denoted as $\mathcal{N}_k \subseteq \mathcal{V}$, contains all clients that are directly connected to k via an edge in $\mathcal{E}$. Each client updates its model by training it locally and aggregating models from its neighbors. The network topology, represented by G, plays a key role in determining how information is shared and how global knowledge is propagated through the network.

A general process of DFL can be divided into the following stages: First, each client k initializes its local model parameters w_0^k. Then, each client trains its local model to further optimize these parameters. Gradient descent is generally used as the core optimization strategy during local training. Specifically, client k performs several epochs of local training using its private dataset P_k to minimize its local loss function $\mathcal{L}(w, P_k)$ as defined in Equation 1.

$$w_t^k \leftarrow w_t^k - \eta \nabla \mathcal{L}(w_t^k, P_k) \tag{1}$$

where the gradient $\nabla \ell(w_t^k, P_k)$ reflects the change in the loss function ℓ with respect to the model parameters w_t^k. The learning rate η controls the step size of the parameter updates at each iteration. By iteratively reducing the loss function, gradient descent effectively guides the model parameters toward a more optimal direction.

After each client k completes the training of its local model, it interacts with its neighbors $\mathcal{N}_k$ and transmits the parameters of the current round of local model training to them, as defined by the network topology G. Simultaneously, client k also receives model updates w_t^j from its neighbors $j \in \mathcal{N}_k$ and follows an aggregation algorithm $A(\cdot)$ to form a new model w_{t+1}^k. The aggeration process can be defined in Equation 2.

$$w_{t+1}^k = A\left(w_t^k, \{w_t^j : j \in \mathcal{N}_k\}\right) \tag{2}$$

By continuously repeating the above processes of local model training and aggregation until the predefined number of rounds is completed, DFL effectively achieves a distributed and collaborative model optimization across the entire network, enhancing both the robustness and privacy of the learning process.

Model poisoning attacks typically cause significant degradation in global model performance by maliciously manipulating model parameters. In the DFL framework, model poisoning attacks typically occur after the client has completed local model training. A malicious client deliberately tampers with its model parameters after training is complete and then sends these tainted parameters to its neighbors and participates in the model aggregation process. In this way, malicious clients are able to gradually contaminate the models of the entire federated learning system, ultimately leading to the degradation of the performance of the global model. The process of the DFL framework with malicious participants is illustrated in Figure1.

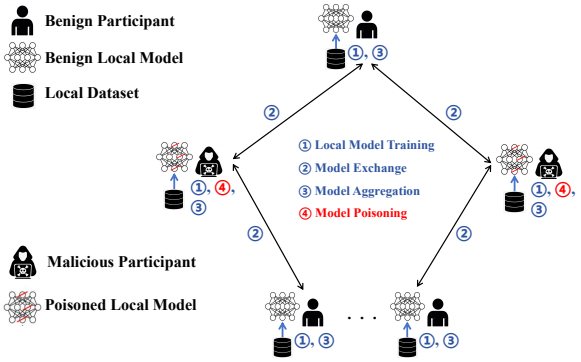


Figure 1: DFL Process with Malicious Participants

Threat Model

Adversary Objective. The primary goal of the attacker is to minimize the performance of the global model in the DFL system by manipulating the updates to the client models under its control. Specifically, the attacker hopes to mislead the learning process of the global model by introducing carefully crafted malicious updates to the model parameters, ultimately leading to significant degradation in model accuracy or deviation in behavior.

Adversary Capability. The attacker has the ability to control a certain percentage of clients and has full access to and modify the local model updates of these clients. The attacker is able to send maliciously modified model parameters to neighboring nodes and participate in model aggregation after each round of training. In addition, the attacker can continuously observe and adjust the attack strategy during the training process to improve the stealth and effectiveness of the attack.

Adversary Knowledge. The attacker only has model information of all malicious participants and has no knowledge of the internal information of other benign clients. This knowledge level assumption is very strict, but it is consistent with the conditions of a realistic attack, which reflects the harmfulness of the designed model poisoning attack discussed in next section.

DMPA Approach

In this section, the general framework for applying the DMPA method in DFL is described, followed by a discussion of the research issues concerning Model Poisoning Attacks in DFL. The section then presents solutions to the problems outlined in the first section.

Method Overview

This workd proposes DMPA, an advanced model poisoning attack specifically designed to target DFL models. As illustrated in Figure 2, DMPA employs a tripartite attack strategy. In contrast to existing model poisoning attacks in FL (e.g., (Shejwalkar and Houmansadr 2021; Baruch, Baruch, and Goldberg 2019; Zhang et al. 2023)), which determine the attack direction based on model similarity, DMPA leverages the maximum eigenvalue of the correlation matrix to identify the optimal poisoning direction. The primary objective is to maximize the training loss of benign models after model aggregation.

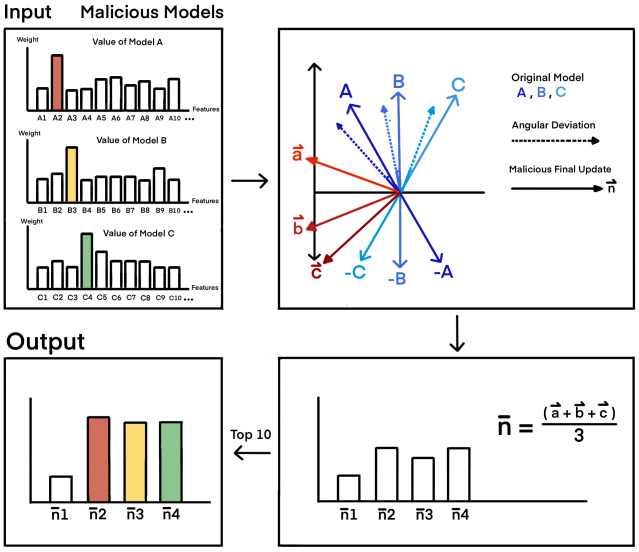


Figure 2: Overview of DMPA Attack Process

As illustrated in Figure 2, DMPA is conducted the attack prior to the exchange of models among interconnected clients. There are three steps for DMPA to achieve the attack: (1) Find the correlation matrix according to the difference of the model parameter matrix, and calculate the eigenvectors of the maximum eigenvalue of the correlation matrix. Use this eigenvector to project on the model parameters to obtain the angular deviation; (2) Add this deviation with the inverse model parameters and the average adjusted model parameters; (3) Extract the uneven values from the average vector and use them to fill the corresponding positions in the average vector, thus maximizing the attack effect. Averaging the posterior vector and filling the original modified vector can prevent the attack effect from decreasing after the model is averaged, and malicious users upload malicious parameters to their connected benign clients for aggregation.

Algorithm 1: DMPA Attack Strategy

Input: A matrix $\mathbf{U} \in \mathbb{R}^{d \times n}$ representing all updates, where each column $\mathbf{u}_i$ is a different update vector.
Output: The modified updates matrix μ_{new}.

Compute the mean vector $\mu = \frac{1}{n} \sum_{j=1}^{n} \mathbf{u}_{i,j,:}$
Center the updates: $\mathbf{V} = \mathbf{U} - \mu$
Compute the covariance matrix: $\mathbf{C} = \frac{1}{n-1} \mathbf{V}\mathbf{V}^{\top}$
Compute the standard deviation: $\mathbf{T} = \sqrt{\text{diag}(\mathbf{C})}$
Compute the correlation matrix: $\mathbf{Y} = \frac{\mathbf{C}}{\mathbf{T}\mathbf{T}^{\top}}$
    where $\oslash$ denotes element-wise division
Compute eigenvalues and eigenvectors of $\mathbf{Y}$: $(\lambda, \mathbf{V}) = \text{eig}(\mathbf{Y})$
Find the principal eigenvalue: $\lambda_{\max} = \max(\lambda)$ and corresponding eigenvector $\mathbf{y}_{\max}$
Compute the projection: $\mathbf{P} = (\mathbf{y}_{\max}^{\top} \mathbf{U}) \cdot \mathbf{y}_{\max}$
Modify the updates: $\mathbf{U}^{\text{new}} = -\mathbf{U} + \mathbf{P}$
Compute the new mean vector: $\mu^{\text{new}} = \frac{1}{d} \sum_{j=1}^{d} \mathbf{U}_{j,:}$
for $i = 1$ to d **do**
    Compute mask $\mathbf{m}_i = \text{select\_top\_k\_params}(\mathbf{u}_i^2, 10)$
    Update mean vector: $\mu^{\text{new}} = \mathbf{m}_i \odot \mathbf{U}_i^{\text{new}} + (1 - \mathbf{m}_i) \odot \mu^{\text{new}}$
end for
return μ^{new}
Function select_top_k_params($\mathbf{vector}, k\%$):
    sorted_indices = argsort($\mathbf{vector}$, descending=True)
    Compute $k = \left\lfloor \frac{\text{length of sorted\_indices} \times k\%}{100} \right\rfloor$
    Initialize mask $\mathbf{m} = \mathbf{0}$ (same shape as $\mathbf{vector}$)
    Set top k indices: $\mathbf{m}[\text{sorted\_indices}_{:k}] = 1$
    return $\mathbf{m}$

Attack Strategy

Algorithm 1 provides the attack pseudocode of DMPA, which explains how malicious clients execute the DPMA attack in DFL. This work defines the model of the received malicious client as $\mathbf{U}$. All malicious client models are transformed into column vectors, represented as $\mathbf{U} = [u_1, u_2, \ldots]$. During the attack phase, a decentralized approach is employed, leading to the computation of a new deviation, denoted as v_{ij}, using Equation 3. These resulting relative offsets collectively constitute the matrix $\mathbf{V}$.

$$v_{ij} = u_{ij} - \bar{u}_j \tag{3}$$

where $\bar{u}_j$ is the average of the j th model parameter.

To ensure the rigor of this analysis, it is essential to compute the overall standard deviation and provide an unbiased estimate. This necessitates the derivation of a correlation matrix. As demonstrated in Equation 4, the correlation matrix $\mathbf{V}$ is determined using the matrix $\mathbf{C}$.

$$\mathbf{C} = \frac{1}{n-1} \mathbf{V}^T \mathbf{V} \tag{4}$$

The matrix $\mathbf{C}$ can be regarded as a covariance matrix, in which each element represents the correlation between subscript corresponding vector groups. However, in this paper,

when finding out the weight of each model for gradient rise, it is necessary to get the weight of each model for gradient rise under independent assumptions. Therefore, the diagonal elements of the matrix $\mathbf{C}$ are extracted and the square root is found. After dimension reduction, the value can reflect the independent weight vector of each model $\mathbf{T}$.

$$\mathbf{Y} = \frac{\mathbf{C}}{\mathbf{T}\mathbf{T}^{\top}} \tag{5}$$

To obtain values characterized by independent weight and correlation deviation, Equation 5 computes the outer product $\mathbf{Y}$ by multiplying the correlation matrix with the independent weight vector. Here, $\mathbf{Y}$ is derived through a linear combination of correlation deviation and independent weight. This vector provides a more accurate representation of the system weight for each parameter column.

$$\mathbf{P} = \left(\mathbf{y}_{\max}^{T}\mathbf{U}\right) \cdot \mathbf{y}_{\max} \tag{6}$$

$$\mu^{\text{new}} = -\mathbf{U} + \mathbf{P} \tag{7}$$

Upon obtaining the independent weight, the eigenvector $\mathbf{y}_{\max}$ associated with the maximum eigenvalue of the weight is computed. This eigenvector is then projected onto the original vector to determine the angular deviation, as described by Equation 6. Subsequently, the original model parameters are adjusted by incorporating the angular deviation, resulting in the final update, as delineated by Equation 7.

Evaluation

Experimental Setup

Datasets. The experiments use CIFAR-10 (Krizhevsky, Hinton et al. 2009), MNIST (Deng 2012) and Fashion-MNIST (Xiao, Rasul, and Vollgraf 2017) as validation datasets, which are widely used in validating model performance. The experiments adopt an independent identically distributed (IID) data partitioning method, which ensures that the data have the same statistical properties. The α parameter is defaulted to 100 as set in ((Shejwalkar and Houmansadr 2021), (Tan et al. 2023), (Li et al. 2024)).

Machine learning models. Three different model architectures are chosen to correspond to different datasets. The batch size is set to 64 for all clients, and the random seed for each client is set to its corresponding ID value. A simple convolutional neural network (CNN) model with Conv2d, BatchNorm2d, 5 Depthwise Conv2d, and fully connected (linear) layers is used for the CIFAR-10 dataset, a model with three fully connected (linear) layers is used for the MNIST dataset, and a simple CNN model with 2 Conv2d and 2 fully connected layers is used for the Fashion-MNIST dataset.

Measurement metrics. The model F1 score is used to assess the attack performance. A lower F1 score indicates a better attack method. All results are the average of the F1 scores of all benign client models after last round of training, which enables a clear assessment of the impact of the MPA attack on the clients in the whole DFL network.

Baseline defenses and attacks. Three of the most effective model poisoning attacks in FL are chosen to attack DFL, namely Lie (Baruch, Baruch, and Goldberg 2019), Min-Sum and Min-Max (Shejwalkar and Houmansadr 2021). In DFL, the modifications of Lie, Min-Max and Min-Sum assume that the malicious clients are colluding, and thus these models can be shared among malicious clients (Li et al. 2023). In addition, four defence methods are chosen; FedAvg, Krum, Trimmed Mean and Median. it is assumed that the malicious client has no knowledge of the benign model and only knows the models of all malicious clients in the current round

- **Lie**: Updates the weights by subtracting the product of the standard deviation of the malicious parameters and the calculated perturbation range from the mean weights(Baruch, Baruch, and Goldberg 2019).
- **Min-Max**: Computes the malicious gradient such that its maximum distance from any other gradient is upper bounded by the maximum distance between any two benign gradients(Shejwalkar and Houmansadr 2021).
- **Min-Sum**: Ensures that the sum of squared distances between the malicious gradient and all benign gradients is upper bounded by the sum of squared distances between any benign gradient and the other benign gradients(Shejwalkar and Houmansadr 2021).

DFL overlay topologies. Three different DFL overlay topologies are employed in the experiments to evaluate the compliance of the proposed attack DPMA in different types of federation.

- **Fully:** Each node is directly connected to every other node.
- **Star:** All nodes are connected through a central node.
- **Ring:** Each node is connected to two neighboring nodes, forming a closed loop.

Training environment. The experiments are conducted using Python 3 and PyTorch, running on an NVIDIA T4 GPU. A total of 10 rounds are run, with each client performing 3 local training epochs in each round. The experiments involve 10 clients, and the DFL experimental programs for all nodes are executed sequentially.

Experimental Results

Compare DMPA with state-of-art model poisoning attacks. In this section, the attack method proposed in this study is compared with the current state-of-the-art model poisoning attack methods, including Lie(Baruch, Baruch, and Goldberg 2019) and Min-Max and Min-Sum (Shejwalkar and Houmansadr 2021). Three topologies (Fully, Star, and Ring) are used for the comparison experiments, and IID data is used for the experiments. The experimental results are based on a configuration of 40% malicious clients and 60% benign clients, and calculate the average F1 scores in the final round (smaller F1 scores represent more effective attacks). The experimental results are shown in Table I. Bolded data indicates the best results.

Firstly, from the experimental results, when the proportion of malicious participants is 40%. The results show that DMPA has a significant attack effect in DFL with different topologies, different aggregation methods, and different

Table 1: Average F1 Score of Benign Clients in DFL with a Configuration Consisting of 40% Malicious Clients for the MNIST, Fashion-MNIST, and CIFAR-10 Datasets in Fully-Connected, Ring, and Star Topologies.

Dataset	Aggregation Rule	Fully				Ring				Star			
		LIE	Min-Max	Min-Sum	DMPA	LIE	Min-Max	Min-Sum	DMPA	LIE	Min-Max	Min-Sum	DMPA
MNIST	Median	0.919523	**0.828144**	0.861094	0.847919	0.864083	0.727043	0.820756	**0.691755**	0.820844	0.821185	0.830490	**0.820525**
	Trimmed mean	0.827509	0.825695	0.843681	**0.801535**	0.864083	0.727043	0.820756	**0.691755**	0.822594	0.819780	0.825661	**0.813209**
	Krum	0.874433	0.811460	0.885670	**0.013312**	0.873979	0.867974	0.840262	**0.740244**	0.819409	0.812599	0.822512	**0.582298**
	Fed_avg	0.825292	0.826278	0.834523	**0.802228**	0.833553	**0.64567**	0.827210	0.672756	0.821639	0.820869	0.825211	**0.815387**
CIFAR-10	Median	0.410982	0.679403	0.706879	**0.044529**	**0.280601**	0.421047	0.446616	0.314470	0.548596	0.665249	0.652866	**0.490103**
	Trimmed mean	0.652813	0.734104	0.732013	**0.016994**	**0.258510**	0.400051	0.395156	0.305964	0.596924	0.636560	0.658856	**0.193741**
	Krum	**0.230500**	0.625591	0.625531	0.569555	**0.372232**	0.545091	0.646702	0.433777	**0.538895**	0.600382	0.594952	0.569471
	Fed_avg	0.674934	0.740961	0.737501	**0.016994**	0.346825	0.440712	0.449766	**0.076015**	0.617962	0.640634	0.652659	**0.213768**
Fashion-MNIST	Median	0.887029	0.892688	0.898451	**0.881820**	0.843861	0.800340	0.885512	**0.773666**	0.883854	0.878469	0.887852	**0.878103**
	Trimmed mean	0.893127	0.885061	0.903730	**0.863626**	0.837478	0.800340	0.885512	**0.773666**	0.887656	0.878575	0.898533	**0.874800**
	Krum	0.875101	0.862932	0.896126	**0.058431**	0.806988	**0.453659**	0.879798	0.767491	0.860227	0.864078	0.884784	**0.597608**
	Fed_avg	0.895660	0.884624	0.897920	**0.861161**	0.880633	**0.754967**	0.894226	0.769304	0.890041	0.873614	0.892970	**0.876514**

datasets, and its F1 scores are all the lowest, indicating that DMPA has the best attack effect, which fully verifies the effectiveness of the method on attack.

Impact of overlay network topology. In fully connected networks, DMPA shows extreme attackability in several experiments. For example, in experiments on the MNIST dataset corresponding to the Krum aggregation method, the CIFAR-10 dataset corresponding to the Median, Trimmed Mean and FedAvg aggregation methods, and the Fashion-MNIST dataset corresponding to the Krum aggregation method, DMPA achieves a score of less than 0.1. This shows that DMPA effectively influences the gradient descent process of 60% benign clients and successfully fills the difference in gradient descent by increasing the loss. This shows that DMPA has been validated for its reasonable design, which can invalidate the gradient descent by increasing and filling in the gradient difference. In contrast, although the other three attack methods achieve the attack by influencing the direction and magnitude of the model update, most of the average F1 scores among the 60% benign clients are greater than 0.8, indicating that their attack effects are more limited.

The overall F1 score of DMPA in the ring network is low and close to the lowest of the other attack methods. The F1 scores of DMPA are all less than 0.8, showing high stability. This may be due to the fact that in ring networks, 60% of the benign clients are buffered to some extent due to the longer chain, mitigating the impact of the attack, which is a major advantage of DFL. As for the other methods, half of them have F1 scores above 0.8, indicating that these methods are not stable enough for model poisoning attacks in the ring structure and are prone to only changing the direction of the model and not effectively affecting other benign clients. This further demonstrates that DMPA exhibits stable and effective attacks that can generally affect benign clients in the DFL ring network. In the Star network, DMPA also shows its advantages, further proving the effectiveness of the method in this paper. Since the results of CIFAR-10 are the most representative for studying the attack ratio among the three datasets, the CIFAR-10 dataset will be selected in the following to further investigate the impact of the malicious client ratio on the overall experimental results.

Impact of increasing the percentage of malicious Participants. Figure 3a to Figure 3c shows the impact of MPA on the F1 score of benign clients after the last round of aggregation in datasets, as the rate of malicious nodes increases from 10% to 60% in the DFL environment. The orange of the dotted line is no attack in each aggregation.

As can be seen, the DMPA method shows the best attack effect in many results to achieve effective attacks. Compared with other methods on different data sets, DMPA has the influence of attacks. However, in different datasets, this paper finds that the more complex the network is, the more effective its influence is. When more network layers are used (for example, CIFAR-10 uses more complex networks), its attack effectiveness is higher than other methods. Obviously, the higher the complexity of the network, the more influence it has. Overall, the method in this paper is more effective than other attack methods at present.

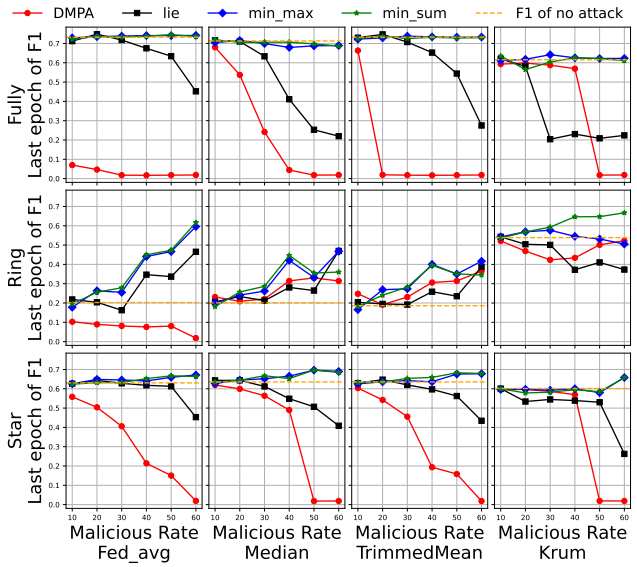
Conclusion and Future Work

This study proposes DMPA, a general framework designed to execute model poisoning attacks within DFL systems. The DMPA framework leverages feature angle deviations to ascertain the most effective attack strategy. In contrast to prior attack methodologies that rely on imprecise or heuristic approaches, DMPA employs the model's numerical properties for its computations, thereby maintaining the model's numerical integrity across iterations. Empirical results indicate that DMPA achieves superior attack efficacy compared to existing state-of-the-art model poisoning techniques, underscoring its substantial practical relevance.

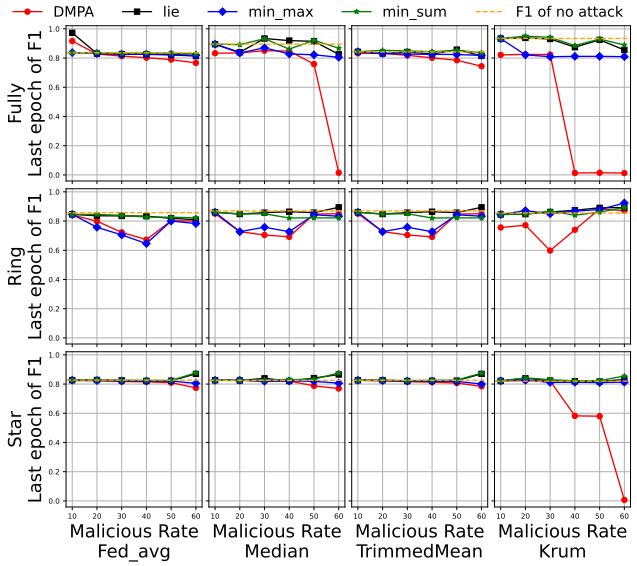
Future research is planned to explore both offensive and defensive dimensions. From an offensive standpoint, existing studies predominantly utilize data distributed in an IID manner, thereby simplifying the attack process. However, the complexity of attacks escalates when data distribution is non-IID. Consequently, future research will focus on devising strategies for effective assaults under non-IID conditions. From a defensive perspective, the proposed DPMA underscores the efficacy of using feature angle deviations as a potent attack vector, providing valuable insights for the development of robust mechanisms to protect against potential threats.

References

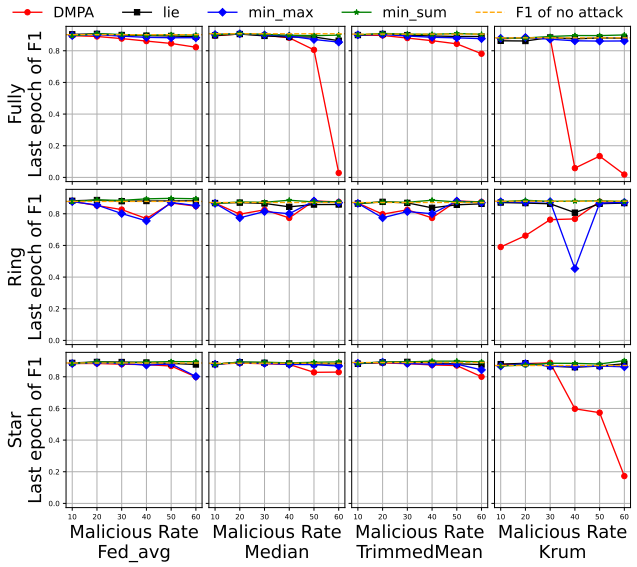
Alazab, M.; RM, S. P.; Parimala, M.; Maddikunta, P. K. R.; Gadekallu, T. R.; and Pham, Q.-V. 2021. Federated learning for cybersecurity: Concepts, challenges, and future direc-



(a) CIFAR-10 Dataset Attack Performance



(b) MNIST Dataset Attack Performance



(c) Fashion-MNIST Dataset Attack Performance

Figure 3: Attack Performance in CIFAR-10, MNIST, and Fashion-MNIST with Different Malicious Clients Rate

tions. *IEEE Transactions on Industrial Informatics*, 18(5): 3501–3509.

Baruch, G.; Baruch, M.; and Goldberg, Y. 2019. A little is enough: Circumventing defenses for distributed learning. *Advances in Neural Information Processing Systems*, 32.

Blanchard, P.; El Mhamdi, E. M.; Guerraoui, R.; and Stainer, J. 2017. Machine learning with adversaries: Byzantine tolerant gradient descent. *Advances in neural information processing systems*, 30.

Cao, X.; and Gong, N. Z. 2022. Mpaf: Model poisoning attacks to federated learning based on fake clients. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 3396–3404.

Deng, L. 2012. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine*, 29(6): 141–142.

Feng, C.; Celdran, A. H.; Baltensperger, J.; Beltran, E. T. M.; Bovet, G.; and Stiller, B. 2023. Sentinel: An Aggregation Function to Secure Decentralized Federated Learning. arXiv:2310.08097.

Feng, C.; Celdran, A. H.; Vuong, M.; Bovet, G.; and Stiller, B. 2024. Voyager: MTD-Based Aggregation Protocol for Mitigating Poisoning Attacks on DFL. *IEEE/IFIP Network Operations and Management Symposium*.

Hallaji, E.; Razavi-Far, R.; and Saif, M. 2022. Federated and transfer learning: A survey on adversaries and defense mechanisms. In *Federated and Transfer Learning*, 29–55. Springer.

Krizhevsky, A.; Hinton, G.; et al. 2009. Learning multiple layers of features from tiny images.

Li, B.; Su, N.; Ying, C.; and Wang, F. 2023. Plato: An open-source research framework for production federated learning. In *Proceedings of the ACM Turing Award Celebration Conference-China 2023*, 1–2.

Li, X.; Wang, N.; Yuan, S.; and Guan, Z. 2024. FedIMP: Parameter Importance-based Model Poisoning Attack Against Federated Learning System. *Computers & Security*, 103936.

Lian, X.; Zhang, C.; Zhang, H.; Hsieh, C.-J.; Zhang, W.; and Liu, J. 2017. Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent. *Advances in neural information processing systems*, 30.

Martínez Beltrán, E. T.; Pérez, M. Q.; Sánchez, P. M. S.; Bernal, S. L.; Bovet, G.; Pérez, M. G.; Pérez, G. M.; and Celdrán, A. H. 2023. Decentralized Federated Learning: Fundamentals, State of the Art, Frameworks, Trends, and Challenges. *IEEE Communications Surveys & Tutorials*, 25(4): 2983–3013.

McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; and y Arcas, B. A. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, 1273–1282. PMLR.

Shejwalkar, V.; and Houmansadr, A. 2021. Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *NDSS*.

Tan, S.; Hao, F.; Gu, T.; Li, L.; and Liu, M. 2023. Collusive model poisoning attack in decentralized federated learning. *IEEE Transactions on Industrial Informatics*.

Xiao, H.; Rasul, K.; and Vollgraf, R. 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*.

Yang, Q.; Liu, Y.; Chen, T.; and Tong, Y. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 10(2): 1–19.

Yin, D.; Chen, Y.; Kannan, R.; and Bartlett, P. 2018. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International conference on machine learning*, 5650–5659. Pmlr.

Yuan, L.; Wang, Z.; Sun, L.; Philip, S. Y.; and Brinton, C. G. 2024. Decentralized federated learning: A survey and perspective. *IEEE Internet of Things Journal*.

Zhang, H.; Yao, Z.; Zhang, L. Y.; Hu, S.; Chen, C.; Liew, A.; and Li, Z. 2023. Denial-of-service or fine-grained control: Towards flexible model poisoning attacks on federated learning. *arXiv preprint arXiv:2304.10783*.